

JAVASCRIPT



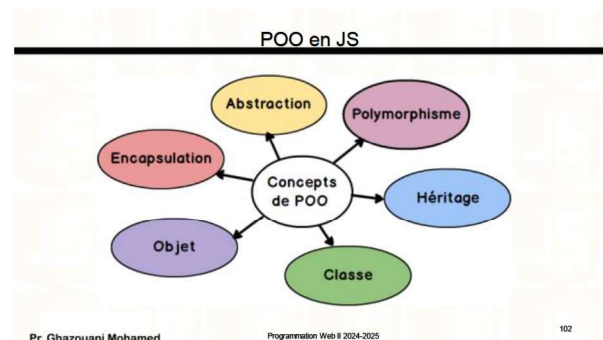
POO EN JS

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

101

101



102

POO en JS

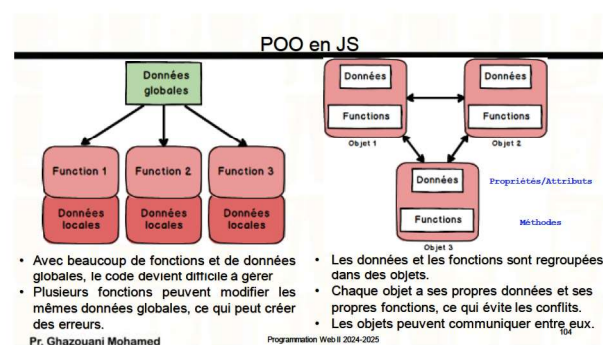
- Java
- Python
- C#
- C++
- JavaScript
- Ruby
- PHP
- Swift
- Visual Basic .NET
- Scala
- Etc.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

103

103



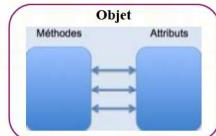
104

POO en JS

Les objets

Le concept d'objet consiste à regrouper dans une même entité:

- des données qu'on nomme des attributs (ou encore des champs)
- des fonctions qu'on nomme méthodes (ou, parfois, fonctions membres)



- Un objet en JavaScript, c'est une boîte qui contient des informations liées ensemble.
- Ces informations sont organisées sous forme de attributs ou propriétés (ce que l'objet possède) et de méthodes (ce que l'objet peut faire).

Seules les méthodes sont habilitées à manipuler ces données.
On dit : les données sont encapsulées dans l'objet

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

105

105

POO en JS

Exemple concret : Une voiture

- Une voiture a des propriétés comme sa couleur, son modèle, et sa marque.
- Elle peut aussi faire des actions (méthodes), comme démarrer ou klaxonner.

```
const Voiture = {
  marque: "Toyota", // Propriété
  modele: "Corolla", // Propriété
  couleur: "Rouge", // Propriété
  prix: 50000, // Propriété
  options: ["GPS, MP4, Caméra de recul"], // Propriété
  getInfo: function() { // Méthode
    return this.marque + " - " + this.modele + " - " + this.couleur + " - " + this.prix + " DH" + " " + this.options;
  },
  klaxonner: function() { // Méthode
    console.log("Tut tut !");
  }
};
// Accéder aux propriétés
console.log(Voiture.marque); // Toyota
// Utiliser les méthodes
console.log(Voiture.getInfo()); // Toyota - Corolla - Rouge - 50000DH - GPS, MP4, Caméra de recul
console.log(Voiture.klaxonner()); // Tut tut !
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

106

106

POO en JS

```
const voiture = {
  marque: "Toyota", // Propriété
  modele: "Corolla", // Propriété
  couleur: "Rouge", // Propriété
  demarrer: function() { // Méthode
    console.log("Vroum vroum ! La voiture démarre.");
  },
  klaxonner: function() { // Méthode
    console.log("Tut tut !");
  }
};
//let marque=voiture.marque;
//let modele=voiture.modele;
//let couleur=voiture.couleur;
//console.log(marque);
let {marque,modele,couleur}=voiture;
console.log(marque);
let {marque:m}=voiture; //console.log(m);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

107

107

POO en JS

Exercice 1 : Créez votre propre objet

Créez un objet étudiant avec les propriétés suivantes :

nom : Votre nom
age : Votre âge
etudier : Une méthode qui affiche "Je suis en train d'étudier !"

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

108

108

POO en JS

Ajouter ou modifier des propriétés

Vous pouvez ajouter ou modifier une propriété après avoir créé l'objet.

```
etudiant.classe = "Informatique"; // Ajouter une propriété
etudiant.age = 22; // Modifier une propriété
console.log(etudiant.classe); // Informatique
console.log(etudiant.age); // 22
```

Supprimer une propriété

On utilise le mot-clé delete pour supprimer une propriété.

```
delete etudiant.age; // Supprime la propriété 'age'
console.log(etudiant.age); // undefined
```

Boucler sur les propriétés

Pour parcourir toutes les propriétés d'un objet, on utilise une boucle for...in.

```
for (let prop in etudiant) {
  console.log(`${prop}: ${etudiant[prop]}`);
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

109

109

POO en JS

Exercice : Calcul du total des salaires

Vous avez un objet JavaScript qui contient des informations sur les salaires de plusieurs employés. Chaque clé de l'objet représente le nom d'un employé, et la valeur associée est son salaire.

Voici un exemple de l'objet fourni :

```
const Employes = {
  "Pierre": 2500,
  "Marie": 5000,
  "Julien": 1200
};
```

Écrire un code JavaScript qui calcule la somme totale des salaires des employés.

```
console.log(employes["Pierre"]); // Affiche 2500
console.log(employes.Pierre); // Affiche 2500
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

110

110

POO en JS

Nous avons vu comment créer des objets individuels. Mais si nous voulons créer plusieurs voitures, il serait fastidieux de répéter le même code.

```
const voitures = {
  marque: "Toyota", // Propriété
  modele: "Corolla", // Propriété
  couleur: "bleu", // Propriété
  prix: 5000, // Propriété
  details: "V6", // Propriété
  getInfo: function() { // Méthode
    return this.marque + " " + this.modele + " " + this.couleur + " " + this.prix + "€ " + this.details;
  },
  klaxonner: function() { // Méthode
    console.log("tut tut !");
  }
};
console.log(voitures.getInfo());

const voitures = {
  marque: "Renault", // Propriété
  modele: "Clio", // Propriété
  couleur: "bleu", // Propriété
  prix: 5000, // Propriété
  details: "V6", // Propriété
  getInfo: function() { // Méthode
    return this.marque + " " + this.modele + " " + this.couleur + " " + this.prix + "€ " + this.details;
  },
  klaxonner: function() { // Méthode
    console.log("tut tut !");
  }
};
console.log(voitures.getInfo());

const voitures = {
  marque: "Renault", // Propriété
  modele: "Clio", // Propriété
  couleur: "bleu", // Propriété
  prix: 5000, // Propriété
  details: "V6", // Propriété
  getInfo: function() { // Méthode
    return this.marque + " " + this.modele + " " + this.couleur + " " + this.prix + "€ " + this.details;
  },
  klaxonner: function() { // Méthode
    console.log("tut tut !");
  }
};
console.log(voitures.getInfo());
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

111

111

POO en JS

Les classes

Les classes sont des 'moules à objets' qui permettent de créer plusieurs objets similaires.

Une classe est une abstraction regroupant des objets ayant les mêmes attributs et les mêmes méthodes.

Un objet est alors une instance de la classe correspondante, et se distingue des autres instances par son identité et la valeur de ses attributs.

Un objet a :

1. Une identité.
2. Des variables définissant son état que l'on appellera attributs.
3. Des sous-programmes gérant son comportement que l'on appellera méthodes.



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

112

112

POO en JS

Une classe n'est pas un objet. Une classe est un patron d'objet.



Classe « Chemise »

Objet 1
Id: 101
Prix: 20\$
Couleur: Bleu
Quantite: 200



Objet 2
Id: 201
Prix: 30\$
Couleur: Rouge
Quantite: 180



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

113

113

POO en JS

```
class Voiture {
  constructor () { //Méthode spéciale pour initialiser les propriétés
    this.marque = "Toyota"; // Propriété
    this.modele = "Corolla"; // Propriété
    this.couleur = "Rouge"; // Propriété
    this.prix = 50000; // Propriété
    this.options = ["GPS", "MP4", "Caméra de recul"]; // Propriété
    this.getInfo = function () { // Méthode
      return this.marque + " - " + this.modele + " - " + this.couleur + " - "
        + this.prix + "DH - " + this.options;
    };
    this.klaxonner = function () { // Méthode
      console.log("Tut tut !");
    };
  };
}
const voiture1 = new Voiture();
console.log(voiture1.getInfo());
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

114

114

POO en JS

```
class voiture {
  constructor () {
    this.marque = "Toyota"; // Propriété
    this.modele = "Corolla"; // Propriété
    this.couleur = "Rouge"; // Propriété
    this.prix = 50000; // Propriété
    this.options = ["GPS", "MP4", "Caméra de recul"]; // Propriété
  }
  getInfo () { // Méthode
    return this.marque + " - " + this.modele + " - " + this.couleur +
      " - " + this.prix + "DH - " + this.options;
  };
  klaxonner () { // Méthode
    console.log("Tut tut !");
  };
};
const voiture1 = new voiture();
console.log(voiture1.getInfo());
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

115

115

POO en JS

Modification d'une propriété

```
const maVoiture = new Voiture();
const taVoiture = new Voiture();

console.log(maVoiture.marque); // Toyota
taVoiture.couleur = "Bleu"; // Modification d'une propriété
console.log(taVoiture.getInfo()); // Toyota - Corolla - Bleue...
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

116

116

POO en JS

```
class Voiture {
  constructor (marque, modele, couleur, prix, options) {
    this.marque = marque; // Propriété
    this.modele = modele; // Propriété
    this.couleur = couleur; // Propriété
    this.prix = prix; // Propriété
    this.options = options; // Propriété
  }
  getInfo() { // Méthode
    return this.marque + " - " + this.modele + " - " + this.couleur + " - " + this.prix + " DH - " + this.options;
  }
  klaxonner() { // Méthode
    console.log("Tut tut !");
  }
}
const voiture1 = new Voiture(
  "Toyota",
  "Corolla",
  "rouge",
  50000,
  ["GPS", "fait ouvrier"]
);
const voiture2 = new Voiture(
  "Renault",
  "Clio",
  "bleu",
  30000,
  ["climatisation"]
);
console.log(voiture1.getInfo());
console.log(voiture2.getInfo());
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

117

117

POO en JS

Exercice : Créer une classe Livre

Partie 1 : Classe de base

Créez une classe Livre avec :

Propriétés :

titre (string)

auteur (string)

anneePublication (number)

Méthode :

decire() → Retourne une chaîne : "[Titre] par [Auteur], publié en [Année]"

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

118

118

POO en JS

L'héritage permet à une classe (enfant) de récupérer les propriétés et méthodes d'une autre classe (parent). C'est comme un arbre généalogique où les enfants héritent des caractéristiques de leurs parents.

Utilité :

- ✓ Éviter la répétition de code (DRY : Don't Repeat Yourself)
- ✓ Créer des hiérarchies logiques (ex : Véhicule → Voiture/Moto/Camion)
- ✓ Polymorphisme : Permet à une même méthode d'avoir des comportements différents en fonction de l'objet sur lequel elle est appelée.

Pr. Ghazouani Mohamed

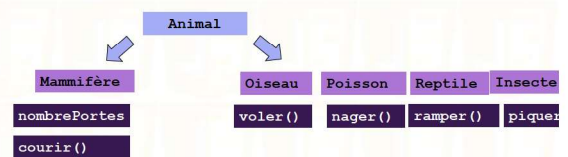
Programmation Web II 2024-2025

119

119

POO en JS

Schéma d'Héritage



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

120

120

POO en JS

Classe Parente : Véhicule

```
class Vehicule {
  constructor(marque, modele, annee, prix) {
    this.marque = marque;
    this.modele = modele;
    this.annee = annee;
    this.prix = prix;
    //this.nbPortes=nbPortes
  }

  demarrer() {
    console.log("Vrrrr... Le véhicule démarre !");
  }

  afficherInfos() {
    return `${this.marque} ${this.modele} ${this.annee} (${this.prix})`;
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

121

121

POO en JS

Classe Enfant : Voiture (Hérite de Véhicule)

```
class Voiture extends Vehicule {
  constructor(marque, modele, annee, prix, nombrePortes) {
    super(marque, modele, annee, prix); // Appelle le constructeur parent
    this.nombrePortes = nombrePortes;
  }

  // Redéfinition de méthode (override / surcharger)
  demarrer() {
    console.log("La voiture démarre avec une clé !");
  }

  // Nouvelle méthode spécifique
  klaxonner() {
    console.log("Tut tut !");
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

122

122

POO en JS

Classe Enfant : Voiture (Hérite de Véhicule)

```
class Voiture extends Vehicule {
  constructor(marque, modele, annee, prix, nombrePortes) {
    super(marque, modele, annee, prix);
    this.nombrePortes = nombrePortes;
  }

  // Redéfinition de méthode (override)
  demarrer() {
    console.log("La voiture démarre avec une clé !");
  }

  // Nouvelle méthode spécifique
  klaxonner() {
    console.log("Tut tut !");
  }
}

class Vehicule {
  constructor(marque, modele, annee, prix) {
    this.marque = marque;
    this.modele = modele;
    this.annee = annee;
    this.prix = prix;
  }

  demarrer() {
    console.log("Vrrrr... Le véhicule démarre !");
  }

  afficherInfos() {
    return `${this.marque} ${this.modele} ${this.annee} (${this.prix})`;
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

123

123

Extends : Déclare l'héritage

super() : Appelle le constructeur parent
=> super(marque, modele, annee, prix)

Méthodes héritées :
afficherInfos() Réutilisées sans modification

Extension : Ajouter de nouvelles fonctionnalités => klaxonner(), faireUnWheeler()

Redéfinition demarrer() : Modifier une méthode existante dans Voiture

Le **polymorphisme** se produit lorsqu'une classe mère a une méthode déclarée, et une ou plusieurs de ses classes filles redéfinissent cette méthode.

POO en JS

Le polymorphisme

```
class Vehicule {
  demarrer() {
    console.log("Démarrage générique d'un véhicule");
  }
}

class Voiture extends Vehicule {
  demarrer() { // Même nom de méthode, comportement différent
    console.log("La voiture démarre avec une clé");
  }
}

class Moto extends Vehicule {
  demarrer() { // Même nom de méthode, comportement différent
    console.log("La moto démarre avec un kick-starter");
  }
}

class Avion extends Vehicule {
  demarrer() { // Même nom de méthode, comportement différent
    console.log("L'avion démarre ses turbines");
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

124

124

POO en JS

Classe Enfant : Moto (Hérite de Véhicule)

```
class Moto extends Vehicule {
  constructor(marque, modele, annee, prix, propriétaireUnique) {
    super(marque, modele, annee, prix);
    this.propriétaireUnique = propriétaireUnique; // Booléen
  }

  // Méthode spécifique
  faireUnWheelie() {
    console.log("La moto lève la roue avant !");
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

125

125

POO en JS

Utilisation

```
const maVoiture = new Voiture("Toyota", "Corolla", 2020, 50000, 5);
const maMoto = new Moto("Yamaha", "MT-07", 2022, 20000, true);

console.log(maVoiture.afficherInfos()); // "Toyota Corolla (2020)"
maVoiture.demarrer(); // "La voiture démarre avec une clé !"
maVoiture.klaxonner(); // "Tut tut !"

console.log(maMoto.afficherInfos()); // "Yamaha MT-07 (2022)"
maMoto.demarrer(); // "Vrrrrr... Le véhicule démarre !" (méthode parent)
maMoto.faireUnWheelie(); // "La moto lève la roue avant !"
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

126

126

POO en JS

Exemple Pratique

- Créez une classe Camion qui hérite de Vehicule avec :
- Une propriété supplémentaire : chargeUtile (en tonnes)
 - Une méthode spécifique : charger()

```
class Camion extends Vehicule {
  constructor(marque, modele, annee, prix, chargeUtile) {
    super(marque, modele, annee, prix);
    this.chargeUtile = chargeUtile;
  }

  charger() {
    console.log(`Chargement de ${this.chargeUtile} tonnes...`);
  }
}

const monCamion = new Camion("Volvo", "FH16", 2023, 70000, 20);
monCamion.charger(); // "Chargement de 20 tonnes..."
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

127

127

POO en JS

Exercice : Créer une classe Livre

Partie 2 : Héritage

Créez une classe LivreNumerique qui hérite de Livre avec :

Propriété supplémentaire :

format (string, ex: "PDF", "ePub")

Redéfinissez la méthode decrire() pour ajouter le format :
"[Titre] par [Auteur], publié en [Année] (Format: [Format])"

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

128

128

POO en JS

Utilisation des modules
C'est quoi un module ?
Un module c'est comme une boîte à outils qui contient des fonctions/variables. On peut :

1. Exporter des outils depuis la boîte
2. Importer des outils dans un autre fichier

Exemple
math.js (la boîte à outils)

```
// On exporte des outils
export function add(a, b) {
  return a + b;
}
```

app.js (le fichier principal)

```
// On importe les outils dont on a besoin
import { add } from './math.js';
console.log(add(2, 3)); // 5
```

car.js

```
export class Car {
  constructor(marque) {
    this.marque = marque;
  }
  klaxonner() {
    console.log("Tut tut !");
  }
}
```

app.js

```
import Car from './car.js';
const maVoiture = new Car("Peugeot");
maVoiture.klaxonner(); // "Tut tut !"
```

Pr. Ghazouani Mohamed Programmation Web II 2024-2025 129

129

POO en JS

car.js

```
export class Car {
  constructor(marque) {
    this.marque = marque;
  }
  klaxonner() {
    console.log("Tut tut !");
  }
}
```

app.js

```
import Car from './car.js';
const maVoiture = new Car("Peugeot");
maVoiture.klaxonner();
```

Pr. Ghazouani Mohamed Programmation Web II 2024-2025 130

130

POO en JS

Restriction CORS : Les navigateurs appliquent des restrictions CORS (Cross-Origin Resource Sharing) pour empêcher les scripts malveillants d'accéder à des ressources d'autres origines sans autorisation explicite.

Origine Différente : Notre fichier HTML est chargé à partir d'une origine null (généralement lorsque vous ouvrez directement un fichier HTML dans votre navigateur en utilisant le protocole file:///). Votre navigateur considère cela comme une origine différente de celle du fichier JavaScript script.js, même s'ils sont sur le même disque dur.

Pr. Ghazouani Mohamed Programmation Web II 2024-2025 131

131

POO en JS

Solutions possibles :

1. Avec VS Code et Live Server :
Si vous utilisez VS Code, installez l'extension **Live Server** et ouvrez votre projet avec. Faites un clic droit sur votre fichier index.html et sélectionnez "Open with Live Server".
2. Avec Node.js et http-server :
Installez http-server avec :
`npm install -g http-server`
Démarrez un serveur local :
`http-server`
Puis, ouvrez `http://localhost:8080` (ou le port affiché).
3. Désactiver temporairement la politique CORS (non recommandé en production)
Si vous testez en local et que vous voulez désactiver temporairement CORS dans Chrome :
Sous Windows : Exécutez Chrome avec :
`chrome.exe --disable-web-security --user-data-dir="C:\chrome_dev"`

Pr. Ghazouani Mohamed Programmation Web II 2024-2025 132

132

POO en JS

En JavaScript (ES6), il y a deux façons d'exporter des éléments d'un module : **export default** ou **export nommé**. Voici la différence expliquée simplement :

export default class Livre (Export Par Défaut)

- À utiliser quand un fichier n'exporte qu'un seul élément principal.
- Importer permet de choisir n'importe quel nom (pas besoin de {}).

Exemple :

Fichier Livre.js

```

export default class Livre {
  constructor(titre, auteur, année) {
    this.titre = titre;
    this.auteur = auteur;
    this.année = année;
  }
}

export function calculerMoyenneLivre(livre) {
  return (livre.année + livre.auteur.length + livre.titre.length) / 3;
}

```

Importation dans un autre fichier

```

import Livre from 'Livre.js';

const monLivre = new Livre('Le Petit Prince', 'Antoine de Saint-Exupéry', 1943);

console.log(monLivre.titre);

```

Pr. Ghazouani Mohamed

133

POO en JS

export class Livre (Export Nommé)

- À utiliser quand vous avez plusieurs éléments à exporter dans un même fichier.
- Importer nécessite d'utiliser le même nom entre {}.

Exemple :

Fichier Livre.js

```

export class Livre {
  constructor(titre, auteur, année) {
    this.titre = titre;
    this.auteur = auteur;
    this.année = année;
  }
}

export function calculerMoyenneLivre(livre) {
  return (livre.année + livre.auteur.length + livre.titre.length) / 3;
}

```

Importation dans un autre fichier

```

import { Livre, calculerMoyenneLivre } from 'Livre.js';

const monLivre = new Livre('Le Petit Prince', 'Antoine de Saint-Exupéry', 1943);

console.log(monLivre.titre);

```

Pr. Ghazouani Mohamed

134

POO en JS

Différence Clé :

	export class Livre (Nommé)	export default class Livre (Par Défaut)
Nombre d'exports	Plusieurs exports par fichier	Un seul export par défaut par fichier
Nom à l'import	Doit correspondre	Peut être différent
Syntaxe à l'import	Utilise {}	Pas de {}

Pr. Ghazouani Mohamed

135